



湖南理工学院
Hunan Institute of Science and Technology

实验报告

课 程： 软件设计模式

班 级： 软件 21-3BF

姓 名： 汪家豪

学 号： 12214801130

日 期： 2024-5-8

实验五 装饰模式实验

一. 实验目的

- 1) 初步了解和掌握装饰模式（Decorate）的类图结构，以及主要的模式角色；
- 2) 理解装饰模式的基本构造，并通过掌握的编程语言，完成实验要求的内容；
- 3) 充分理解和掌握装饰模式如何利用对象的聚合功能，实现对象功能的动态扩展。

二. 实验内容

- 1, 实验场景的设计如下：

假设有三种型号的坦克型号（T50、T75、T90）每种坦克均可以进行三种不同的功能扩展：1) 红外线夜视功能 A；2) 水陆两栖功能 B；3) 卫星定位功能 C。

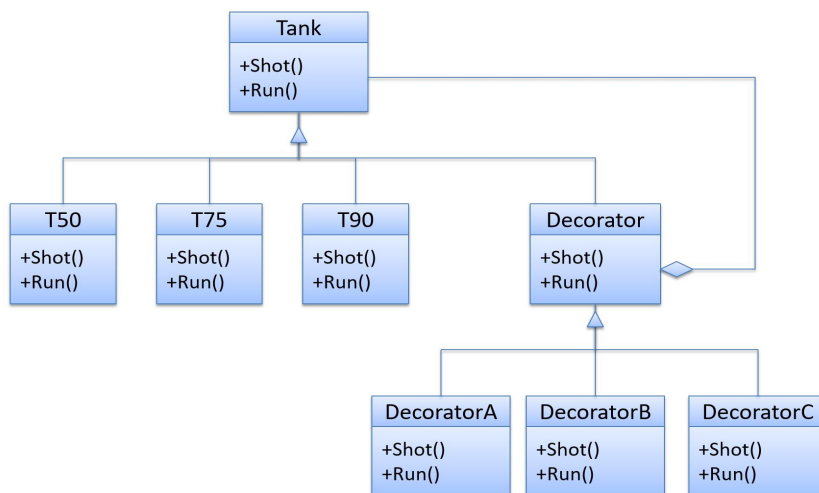
- 2, 利用面向对象程序设计，引入装饰模式，并对模式的不同角色及其类进行定义，实现在不同作战环境下各种型号坦克可以动态的扩展上述代号为 A、B、C 的三种功能。

- 3, 通过对装饰模式类图结构的理解，编程实现该模式的一般化程序代码。

三. 实验过程

模式类图

利用装饰模式实现坦克3种功能的扩展



1. 抽象构建类

首先构建 component 类，是一个抽象类，在本例中，tank 类就是这个抽象类，它声明了 Tank 对象的两个基本方法，Shot 和 Run

```
10 个用法 7 个继承者
public abstract class Tank
{
    5 个用法 7 个实现
    public abstract void Shot();
    5 个用法 7 个实现
    public abstract void Run();
}
```

2. 具体构建类

然后在本例中还有三个继承 Tank 类的具体构建类，代表不同的具体坦克型号，这里以 T50 为例，重写了 Shot 和 Run 方法，不同的坦克型号有不同的射击方法和速度。

```
1 个用法
public class T50 extends Tank{
    5 个用法
    @Override
    > public void Shot() { System.out.println("T50坦克平均每秒射击5发子弹"); }
    5 个用法
    @Override
    > public void Run() { System.out.println("T50坦克平均每时运行30公里"); }
}
```

3. 抽象装饰类

抽象装饰类是装饰模式的核心设计，它是抽象构件类的子类，并抽象构件类对象聚合进来，实现了在抽象构件类中定义的方法，只不过仅仅在调用构件对象的原有业务方法，具体的装饰由子类完成。

在本例中，下图是一个装饰模式下的抽象装饰类，它继承了 Tank 的类。装饰模式是一种结构型设计模式，它允许向一个现有的对象添加新的功能，同时不

改变其结构。

在这个抽象装饰类中，它包含了一个私有的 Tank 对象作为组合关系。通过组合的方式，抽象装饰类可以访问和使用 Tank 对象的方法。

抽象装饰类重写了 Shot()和 Run()方法，并在这些方法中调用了 tank 对象的相应方法。这样，通过继承抽象装饰类并添加新的功能，可以在不改变 Tank 类的基础上为 Tank 对象添加附加的行为。

```
3 个用法 3 个继承者
public abstract class Decorator extends Tank //继承
{
    3 个用法
    private Tank tank; //对象组合
    3 个用法
    public Decorator(Tank tank)
    {
        this.tank = tank;
    }
    5 个用法 3 个重写
    > public void Shot() { tank.Shot(); }
    5 个用法 3 个重写
    public void Run()
    {
        tank.Run();
    }
}
```

4. 具体装饰类

这是一个装饰模式下的具体装饰类 DecoratorA，它继承自抽象装饰类 Decorator。具体装饰类用于实现具体的功能扩展，并在运行时动态地为对象添加新的行为。

在这个具体装饰类中，重写了 Shot()方法并添加了红外功能的扩展，然后调用了父类的 Shot()方法。这样，当 Shot()方法被调用时，会先执行具体装饰类 DecoratorA 中的红外功能扩展，然后再调用基础 Tank 对象的 Shot()方法。

另外，在 `Run()`方法中，具体装饰类并没有添加新的功能，而是直接调用了父类的 `Run()`方法，保持了原有的行为。

通过创建具体装饰类并将其组合到基础对象中，可以动态地为对象添加额外的功能，而无需改变其结构。这种方式使得系统更加灵活和可扩展。

```
public class DecoratorA extends Decorator
{
    1 个用法
    > public DecoratorA(Tank tank) { super(tank); }
    5 个用法
    public void Shot()
    {
        // 功能扩展---红外功能
        System.out.println("增加红外线功能");
        super.Shot();
    }
    5 个用法
    > public void Run() { super.Run(); }
}
```

这是另一个装饰模式下的具体装饰类 `DecoratorB`，它也继承自抽象装饰类 `Decorator`。与 `DecoratorA` 类似，`DecoratorB` 也用于实现具体的功能扩展。

在这个具体装饰类中，重写了 `Shot()`方法并添加了水陆两栖功能的扩展，然后调用了父类的 `Shot()`方法。这样，当 `Shot()`方法被调用时，会先执行具体装饰类 `DecoratorB` 中的水陆两栖功能扩展，然后再调用基础 `Tank` 对象的 `Shot()`方法。

```

public class DecoratorB extends Decorator
{
    1 个用法
    public DecoratorB(Tank tank) { super(tank); }

    5 个用法
    public void Shot()
    {
        // 功能扩展---- 水陆两栖功能
        System.out.println("增加水陆两栖功能");
        super.Shot();
    }

    5 个用法
    public void Run() { super.Run(); }
}

```

这是另一个装饰模式下的具体装饰类 `DecoratorC`，它也继承自抽象装饰类 `Decorator`。与之前的具体装饰类类似，`DecoratorC` 用于实现具体的功能扩展。

在这个具体装饰类中，重写了 `Shot()` 方法并添加了卫星定位功能的扩展，然后调用了父类的 `Shot()` 方法。这样，当 `Shot()` 方法被调用时，会先执行具体装饰类 `DecoratorC` 中的卫星定位功能扩展，然后再调用基础 `Tank` 对象的 `Shot()` 方法。

与之前的具体装饰类一样，在 `Run()` 方法中，具体装饰类并没有添加新的功能，而是直接调用了父类的 `Run()` 方法，保持了原有的行为。

通过创建多个具体装饰类并将它们组合到基础对象中，可以实现逐渐增加对象功能的继承关系。这样，可以根据需求逐步地添加新的功能，而不会影响到基础对象的结构。

```

2 个用法
public class DecoratorC extends Decorator
{
    1 个用法
    > public DecoratorC(Tank tank) { super(tank); }

    5 个用法
    public void Shot()
    {
        // 功能扩展--- 卫星定位功能
        System.out.println("增加卫星定位功能");
        super.Shot();
    }

    5 个用法
    > public void Run() { super.Run(); }
}

```

主程序

这是装饰模式的主程序 Program，在 main 方法中创建了一个 T50 坦克对象，并通过一系列装饰类为该坦克对象添加不同的功能。

首先创建了一个 DecoratorA 对象 da，它包装了 T50 坦克对象并添加了红外功能。然后创建了一个 DecoratorB 对象 db，它包装了 da 对象并添加了水陆两栖功能。最后创建了一个 DecoratorC 对象 dc，它包装了 db 对象并添加了卫星定位功能。

在最后调用了 dc 的 Shot()和 Run()方法，触发了装饰类中相应的功能扩展代码。由于装饰类之间的层层嵌套，每个装饰类都为 T50 坦克对象添加了新的功能，而不会改变原有的 T50 坦克类的结构。

这种方式使得可以在运行时动态地为对象添加新的功能，而无需修改原始类的代码，实现了开闭原则和单一职责原则。

```

public class Program {

    public static void main(String[] args) {
        // TODO 自动生成的方法存根
        Tank tank = new T50();
        DecoratorA da = new DecoratorA(tank); // 且有红外功能
        DecoratorB db = new DecoratorB(da); // 且有红外和水陆两栖功能
        DecoratorC dc = new DecoratorC(db); // 且有红外、水陆两栖和卫星定位功能
        dc.Shot();
        dc.Run();
    }
}

```

四. 调试和运行结果

增加卫星定位功能

增加水陆两栖功能

增加红外线功能

T50坦克平均每秒射击**5**发子弹

T50坦克平均每时运行**30**公里

五. 实验总结

装饰模式是一种结构型设计模式,它通过动态地将责任附加到对象上来扩展对象的功能。该模式提供了一种灵活的替代继承的方式,可以避免受限于固定的类层次结构。装饰模式的实践有助于深入理解面向对象设计的思想,提高代码的可维护性和可扩展性。